

QUDA multigrid 算法实现

从近零模、Galerkin 粗化到 MPI 递归 V-cycle 的可复现逻辑

源码分析汇报

QUDA 1.1.0 参考源码快照

2026 年 8 月 28 日

本版报告的验收目标：读者可脱离 QUDA 重建同一算法

数学等价

给定同一细格算子 D 、零空间 B 和聚合规则，能够在 CPU、GPU 或任意语言中构造同一 P/R 、 $D_c = RDP$ 与 Schur 补系统。

控制流等价

伪代码明确写出 setup、batch、正交化、pre/post smooth、残差选择、递归、prepare/reconstruct 和停止条件。

实现可迁移

先给参考实现；再把 QUDA 的 checkerboard、halo、atomic、MMA、低精度和 cycle wrapper 映射回抽象接口。

边界声明

本报告确证“源码实现了哪些算法步骤”，并给出可执行的复现规范；本次没有启动 CUDA/MPI，因此不虚报特定硬件的收敛、吞吐或加速比。



先固定“算法合同”：只需要四类抽象接口

数据与线性代数

接口	必须提供的语义
Field	复数旋量/矩阵，支持 full 与 parity subset
$D\psi$	细格 residual operator，含边界条件与质量归一化
$\langle u, v \rangle$	全局或局部复内积；范数为 $\sum u ^2$
P, R	同一聚合基上的 prolong/restrict

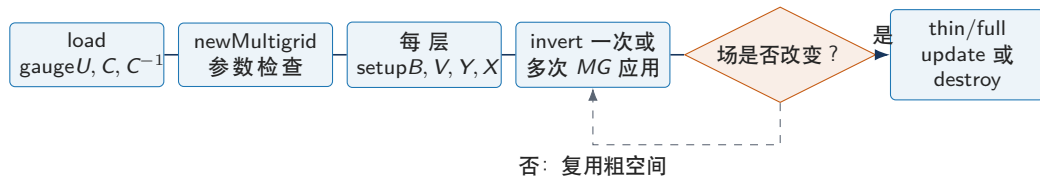
实现层必须保持的不变量

1. 粗点映射对每个细点唯一，且 inverse map 覆盖整个聚合；
2. setup 中 (V) 的列在每个 aggregate 内正交归一；
3. $R = P^\dagger$ （本报告的 aggregate 路径）；
4. coarse apply 与显式 (RDP) 的动作一致。

可移植策略

任何后端都可先用“逐粗基向量 apply $P \rightarrow D \rightarrow R$ ”构造 D_c ，通过验证后再替换为 UV/VUV 融合核。

总体执行链不是黑盒：API、setup、solve、update 各自有边界



构造时

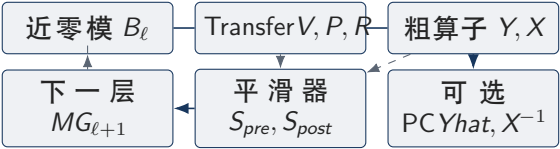
公共入口创建细格 residual/smoothen Dirac, 再以 (B) 创建 (MG(0)); 非粗层由 'reset()' 创建 Transfer、粗场、粗 Dirac、下一层与 coarse solver。

来源：公共入口：lib/interface_quda.cpp:2853-2944; 更新：lib/interface_quda.cpp:2946-3053; 递归 reset：lib/multigrid.cpp:72-197。

更新时

thin update 只更新 Dirac 所引用的 gauge/clover 与质量；full update 重新创建细格 Dirac 并递归 'reset(refresh=true)', 必要时刷新零空间。

每个非底层 MG_ℓ 的状态：六个对象足以复现运行期



状态	用途
B_ℓ	setup 的输入/输出，下一层的候选零空间
V_ℓ	aggregate-local 的列正交基
Y, X	粗 hopping 与 local term
$Yhat, X^{-1}$	even/odd 预条件 dslash
smoother	负责局部高频误差
coarse solver	递归 V-cycle 或 底层 Krylov

复现顺序

先实现 $B \rightarrow V \rightarrow P/R \rightarrow D_c$ ；再实现 $X^{-1}, Yhat$ ；最后接入 smooth/recursive solve。每一步都能单独与显式矩阵动作对比。

来源：成员与层级参数：include/multigrid.h:75-217, 260-340；创建顺序：lib/multigrid.cpp:94-186。

维数协议：先缩小时空，再重组自旋与颜色自由度

$$L_c^\mu = \frac{L_f^\mu}{b_\mu}, \quad N_c^{\text{color}} = N_{\text{vec}}, \quad N_c^{\text{spin}} = \begin{cases} N_f^{\text{spin}}/b_s, & b_s > 0 \\ 2, & b_s = 0 \text{ (staggered)}. \end{cases}$$

对象

维数/布局

复现要点

聚合容量约束

对 aggregate transfer，细空间的局部容量是

$$C_{agg} = \left(\prod_\mu b_\mu\right) N_f^{\text{color}} \times \begin{cases} b_s, & b_s > 0, \\ 1/2, & b_s = 0, \end{cases}$$

必须有 $N_{\text{vec}} \leq C_{agg}$ 。

两个容易混淆的事实

‘V’ 的存储颜色数可写成细颜色乘 ‘Nvec’；粗 GaugeField 的颜色维则是 $N_c^s N_c^c$ 。两者通过 spin map 和 tile 索引连接，不能仅凭扁平数组长度猜测矩阵形状。

奇偶与线性索引：数学上的坐标必须和存储的 checkerboard 对齐

$$p(x) = \left(\sum_{\mu=0}^3 x_{\mu} \right) \bmod 2, \quad x_c^{\mu} = \left\lfloor \frac{x_f^{\mu}}{b_{\mu}^c} \right\rfloor$$

两张 map

`fine_to_coarse[i]`: 细格

parity-ordered 线性索引 $i \rightarrow k_c$ 。

`coarse_to_fine[j]`: 按 (k_c, i_f) 排序得到的逆向遍历表；它让一个 coarse block 的细点连续被处理。

```
1 for each fine linear index i:
2     x_f, p_f = lattice_coordinate_and_parity(i)
3     x_c = floor_div(x_f, geo_block_size)
4     k_c = parity_ordered_index(x_c, parity(x_c))
5     fine_to_coarse[i] = k_c
6
7 for each i:
8     pairs.append((fine_to_coarse[i], i))
9 sort(pairs) # group by coarse
10             site
11 coarse_to_fine = [pair.fine_index for pair in pairs]
12
13 if spin_block_size == 0: # staggered
14     spin_map[0][even] = 0
15     spin_map[0][odd] = 1
16 else:
17     for s_f in fine_spin:
18         spin_map[s_f][p] = floor_div(s_f,
19                                         spin_block_size)
```

单层 setup 总览：每一步都有输入、状态变化和可验证输出

```
1  setup_level(level, U, C, B, cfg):
2      validate_geometry_and_capacity(U.shape, cfg.geo_block_size,
3                                     cfg.spin_block_size, cfg.n_vec)
4      if cfg.compute_null:
5          B = initialize_or_load_or_generate(B, cfg.null_source)
6          B = generate_null_vectors(B, level, cfg)
7
8      map = build_geometry_maps(U.shape, cfg.geo_block_size)
9      spin_map = build_spin_map(U.n_spin, cfg.spin_block_size)
10     V = block_orthonormalize(B, map, spin_map, cfg)
11     P = make_prolongator(V, map, spin_map)
12     R = make_restructor(V, map, spin_map)      #  $R = P^\dagger$ 
13
14     (Y, X) = build_galerkin_operator(U, C, V, map, spin_map, cfg)
15     if cfg.needs_pc:
16         Xinv = inverse_local_blocks(X)
17         Yhat = build_preconditioned_links(Y, Xinv)
18
19     coarse = allocate_next_level(L_f / geo_block_size,
20                                  Nvec=cfg.n_vec, Nspin=coarse_spin)
21     verify_transfer_and_operator(P, R, D, (Y, X), cfg)
22     return Level(B, V, P, R, Y, X, Xinv, Yhat, coarse)
```


几何检查不是装饰: QUDA 会为不可用 block 反复减半

```
1 for d in dimensions 0..3:
2     while b[d] > 0:
3         coarse_d = L_f[d] / b[d]
4         invalid = (d == 0 and L_f[0] == b[0])
5                 or ((coarse_d + 1) % 2 == 0)
6                 or (coarse_d * b[d] != L_f[d])
7         if not invalid:
8             break
9         warn_or_record(d, b[d])
10        b[d] = b[d] / 2
11    if b[d] == 0:
12        fail("dimension cannot be blocked")
13
14    aggregate = product(b)
15    if spin_block == 0:
16        capacity = aggregate * N_f_color / 2
17    else:
18        capacity = aggregate * N_f_color * spin_block
19    assert Nvec <= capacity
20    assert 1 < aggregate <= 1024 and aggregate % 2 == 0
21
```

为什么要避免奇数粗维

QUDA 当前 checkerboard 索引要求粗 (x) 方向可分 parity; 因此粗维奇偶和整除性会影响 'OffsetIndex'、coarse-to-fine 以及 parity halo。

实现选择

若目标是数学复现, 可直接拒绝非法配置并要求用户修正; 若目标是行为复现, 则实现同样的逐维减半与 warning, 再记录最终 block。

近零模生成 (1/2): setup solver 的参数决定 (B) 如何变平滑

```
1 generate_null_vectors(B, level, cfg, refresh=false):
2     sp.maxiter = refresh ? cfg.setup_maxiter_refresh[level]
3                     : cfg.setup_maxiter[level]
4     sp.tol = cfg.setup_tol[level]
5     sp.use_init_guess = YES
6     sp.delta = 1e-1
7     sp.inv_type = cfg.setup_inv_type[level]
8     sp.precision = cfg.cuda_prec_sloppy
9     sp.residual_type = L2_RELATIVE
10    sp.compute_null_vector = YES
11
12    batch = cfg.n_vec_batch[level]
13    require len(B) % batch == 0
14    for setup_iter in 1 .. cfg.num_setup_iter[level]:
15        if cfg.pre_orthonormalize:
16            global_gram_schmidt(B)
17
18        for first in range(0, len(B), batch):
19            B_i = B[first:first+batch]
20            if cfg.setup_type == TEST_VECTOR_SETUP:
21                rhs = copy(B_i); x = zero_like(B_i)
22            else:
23                x = copy(B_i); rhs = zero_like(B_i)
```

近零模生成 (2/2): CG、GCR、MG 三条内部求解路径不能混写

CG / CA-CG

setup solver 不是直接作用于 D , 而是包装 'DiracMdagM', 即以 $D^\dagger D$ 为求解矩阵。
CA-CG 另外设置 Chebyshev/CA basis 参数。

普通 Krylov

GCR、BiCGStab-L 等由 'Solver::create' 直接以 'matSmooth' 为矩阵; 若输入配置带 Schwarz, 源码在此阶段先暂时关闭它。

MG setup 自举

当 setup_inv_type=MG:

setup solver = GCR, $K = MG_\ell$, 每轮最多

先保证本层 smoother 存在, 再用本层 MG 作为 GCR 的 preconditioner。

递归更新

generate_all_levels=true: 向下层重新生成; 否则先 $B_{\ell+1} = R_\ell B_\ell$, 再重建下一层 Transfer。自举完成后恢复 Schwarz/halo 精度并释放临时 solver。

来源: solver 分支: lib/multigrid.cpp:1335-1363; MG 自举与向下传播: lib/multigrid.cpp:1429-1466; Schwarz 临时关闭: lib/multigrid.cpp:1321-1333。
源码分析汇报

全局正交化与块正交化是两层不同约束

全局正交化（可选）

作用域是整个 local/global lattice 的 B_i :

$$B_i \leftarrow B_i - \sum_{j < i} \langle B_j, B_i \rangle B_j, \quad B_i \leftarrow B_i / \|B_i\|.$$

由 `pre_orthonormalize`、`post_orthonormalize` 控制，内积可能跨 MPI reduction。

块正交化（Transfer 构造必需）

每个 aggregate、每个 coarse-spin/chirality block 单独执行 Gram-Schmidt；它保证的是 $V^\dagger V \approx I$ 的局部关系，而不是不同 aggregate 之间的全局正交。

全局 B_i : $\langle B_j, B_i \rangle$

按 aggregate
聚类细点

局部 V_i : 内
积与归一

不要用全局 $B^\dagger B = I$ 替代块正交

Galerkin transfer 的局部列基按 aggregate 存储；只做全局正交并不能使同一 aggregate 内的列成为 $P^\dagger P$ 的单位块。

来源：全局正交化：lib/multigrid.cpp:1369-1381, 1415-1427；块正交化：

include/kernels/block_orthogonalize.cuh:141-253

QUADA multigrid 复现手册

块正交化伪代码 (1/2): 按 aggregate 和向量批次复现 kernel 逻辑

```
1 block_orthonormalize(B, geo_map, spin_map, n_block_ortho):
2     V = allocate_matrix_field(n_vec, fine_spin, fine_color)
3     for each coarse site q:
4         for each coarse chirality h:
5             for repeat in 0 .. n_block_ortho-1:
6                 for j0 in 0, mVec, 2*mVec, ... n_vec-mVec:
7                     v[0:mVec] = load(B if repeat == 0 else V,
8                                     q, aggregate(q), h, j0:j0+mVec)
9
10                    # Orthogonalize the current tile against previous vectors.
11                    for i in 0 .. j0-1:
12                        vi = load(V, q, aggregate(q), h, i)
13                        for m in 0 .. mVec-1:
14                            coeff[m] = sum_{x in aggregate(q)}
15                                inner(vi(x), v[m](x))
16                        coeff = reduce_within_aggregate(coeff)
17                        for m in 0 .. mVec-1:
18                            v[m] = v[m] - coeff[m] * vi
19
20                    local_block_orthogonalize_and_normalize(v)
21                    store(V, q, aggregate(q), h, j0:j0+mVec, v)
22
```

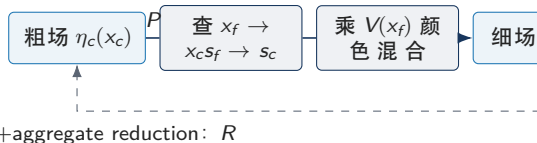
块正交化伪代码 (2/2): tile 内再正交, 避免只做半个 Gram-Schmidt

```
1 local_block_orthogonalize_and_normalize(v[0:mVec]):
2     for m in 0 .. mVec-1:
3         for i in 0 .. m-1:
4             c = reduce_{x in aggregate}(inner(v[i](x), v[m](x)))
5             v[m] = v[m] - c * v[i]
6             n2 = reduce_{x in aggregate}(norm2(v[m](x)))
7             if n2 <= 0: fail("zero null vector in aggregate")
8             v[m] = v[m] / sqrt(n2)
9
10 two_pass_block_ortho(B, cfg):
11     V = block_orthonormalize(B, ..., cfg.n_block_ortho)
12     if cfg.block_ortho_two_pass and V.precision < SINGLE:
13         scale = abs_max(V)
14         if 1.05 * scale < 1:
15             V.set_storage_scale(1.05 * scale)
16             V = block_orthonormalize(B, ..., cfg.n_block_ortho)
17     return V
18
```

(P/R) 的数学定义：局部基列把粗自由度展开到细网格

$$(P\psi_c)_{s_f c_f}(x_f) = \sum_{a=0}^{N_c^c-1} V_{s_f c_f, a}(x_f) \psi_{c, s_c(s_f, p_f), a}(x_c(x_f)).$$

$$(R\psi_f)_{s_c a}(x_c) = \sum_{x_f \in A(x_c)} \sum_{s_f, c_f} V_{s_f c_f, a}^*(x_f) \psi_{f, s_f c_f}(x_f).$$



局部单位性

若每个 aggregate 内 $V^\dagger V = I$, 则对粗场

$$(RP\eta_c)(x_c) = \eta_c(x_c),$$

这是 'coarse span' 检查的理论基准。

来源: P/R 入口: lib/transfer.cpp:259-328; kernel 旋转: include/kernels/prolongator.cuh:65-132, include/kernels/restrictor.cuh:87-124。

源码分析汇报

奇偶 subset

full field 计算两个 parity; single-parity 由 'Transfer::setSiteSubset' 固定目标 parity。aggregate 路径不能把只有单 parity 的 (V) 错当成可作用于 full field 的基。

P/R 参考实现：先保证累加方向、清零和 parity 完全正确

```
1 prolongate(out_f, in_c):
2     for each fine site x_f in requested parity subset:
3         q = geo_map[x_f]
4         p = parity(x_f)
5         for s_f, c_f:
6             s_c = spin_map[s_f][p]
7             out_f[x_f,s_f,c_f] = 0
8             for a in coarse_colors:
9                 out_f[x_f,s_f,c_f] += V[x_f,s_f,c_f,a]
10                    * in_c[q,s_c,a]
11
12 restrict(out_c, in_f):
13     fill(out_c, 0)
14     for each fine site x_f in requested parity subset:
15         q = geo_map[x_f]
16         p = parity(x_f)
17         for s_f, c_f, a:
18             s_c = spin_map[s_f][p]
19             out_c[q,s_c,a] += conj(V[x_f,s_f,c_f,a])
20                    * in_f[x_f,s_f,c_f]
21
22     return out_c
```


最稳妥的 Galerkin 参考实现：对每个粗基列执行 $P \rightarrow D \rightarrow R$

```
1 build_galerkin_reference(D, P, R, coarse_shape):
2     Dc = zero_matrix_or_operator(coarse_shape)
3     for each coarse source basis index beta=(q,s_c,a):
4         e_c = unit_vector(beta)
5         e_f = P(e_c)
6         y_f = D(e_f)
7         column = R(y_f)
8         Dc[:, beta] = column
9     return split_into_Y_and_X(Dc)
10
11 split_into_Y_and_X(Dc):
12     for each output q and input q2:
13         if q2 == q: X[q] += Dc[q,q2]
14         elif q2 == q + unit_direction(mu): Y_fwd[mu,q] = Dc[q,q2]
15         elif q2 == q - unit_direction(mu): Y_back[mu,q] = Dc[q,q2]
16         else: preserve_explicit_long_range_term_or_fail()
17
```

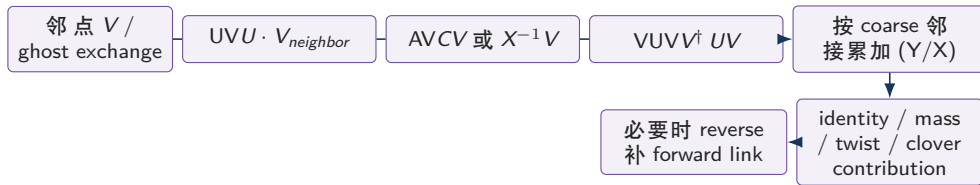
为何这是“任何形式”都能复现的版本

它不依赖 CUDA thread/block、GaugeField

结构约束

标准 Wilson/Clover aggregate 将所有跨

QUDA 的快速 Galerkin 图: UV/VUV 只是显式 (RDP) 的融合与复用



$$D_c = RDP, \quad Y_{q \rightarrow q'} = \sum_{x \in A_q, y \in A_{q'}} V^\dagger(x) H(x, y) V(y), \quad X_q = \sum_{x, y \in A_q} V^\dagger(x) D(x, y) V(y).$$

Wilson/Clover

forward projector 使用 $1 - \gamma_\mu$, backward 使用 $1 + \gamma_\mu$; 代码把 spin 对角和非对角项分别累加到 coarse spin block。

来源: UV/VUV 调度: lib/coarse_op.cuh:1043-1062, 1166-1215, 1269-1323; Wilson projector: include/kernels/coarse_op_kernel.cuh:1067-1138; 应用时 $-\kappa$: include/kernels/dslash_coarse.cuh:292-324。

存储约定

外部 hopping 的 Y 通常不含应用时的 $-\kappa$; dim_collapse 在粗 dslash 汇总后乘 $-\kappa$ 。内部项已在写入 X 前乘相应系数。